

Lecture 1: Introduction

41.601
Agentic AI

etienne@cmkl.ac.th

Class Organization

- Canvas as Learning Platform
  - Lecture files, lab code, lecture recordings
  - Announcements
- MS Teams for recordings and group chat
  - Lectures are being recorded
  - Labs are NOT being recorded (-> come to class)
- Assignments (tba):
  - One assignment after the 4th lecture (7 November)
  - One assignment/presentation at the end (28 November)
- (No lecture on 14 November)



Content

- Intro
- LLMs Overview
- Prompting
- Memory & Tools
- Lab



Introduction?

- Anybody new?
- What do you know about LLMs so far?



What is an AI agent?

- A system that uses an LLM to decide what actions to take, executes those actions, observes the results, and repeats
- **Key distinction**: LLM answers questions; agent accomplishes goals

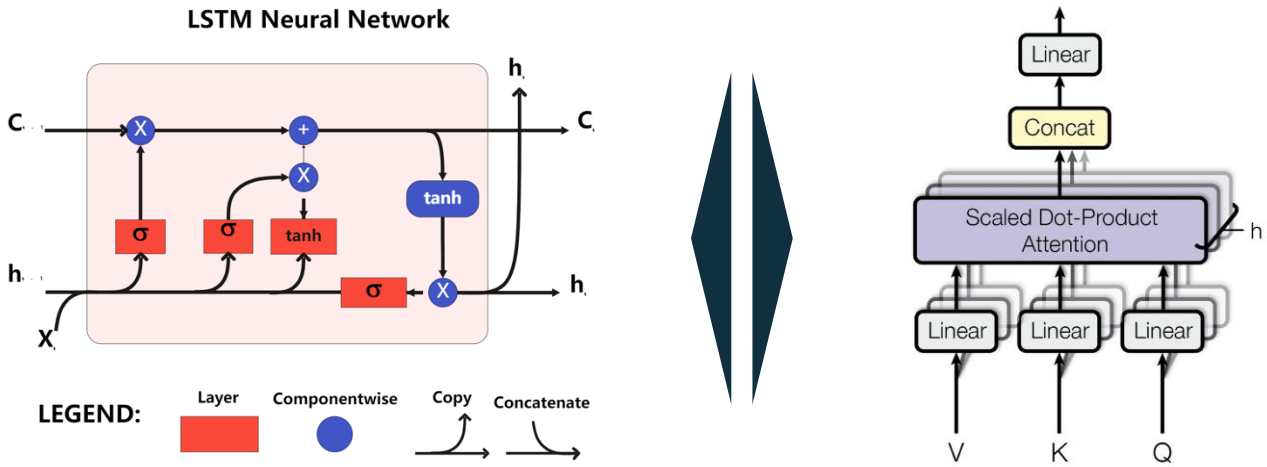


Content

- Intro
- LLMs Overview
- Prompting
- Memory & Tools
- Lab



RNNs vs. Transformers



- Sequential processing: Words processed one by one (left to right)
- The "Vanishing Gradient": Forgetting the beginning of a sentence by the time the end is reached
- Bottleneck: Slow training because you can't parallelize



“Attention is all you need”

- Vaswani et al. (2017): "Attention Is All You Need"
- The core innovation: Self-Attention
- Goal: Process the entire sentence simultaneously (Parallelization)

https://arxiv.org/abs/1706.03762

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

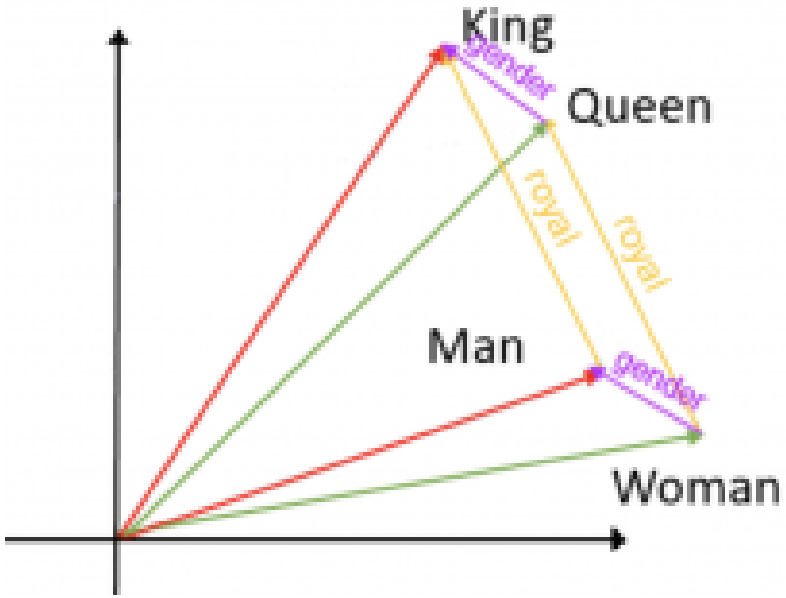
Tokenization

- LLMs don't see words; they see numbers (tokens)
- Sub-word tokenization (e.g., "Transforming" → "Transform" + "ing")
- Vocabulary size: Why it determines the model's "knowledge" of languages.



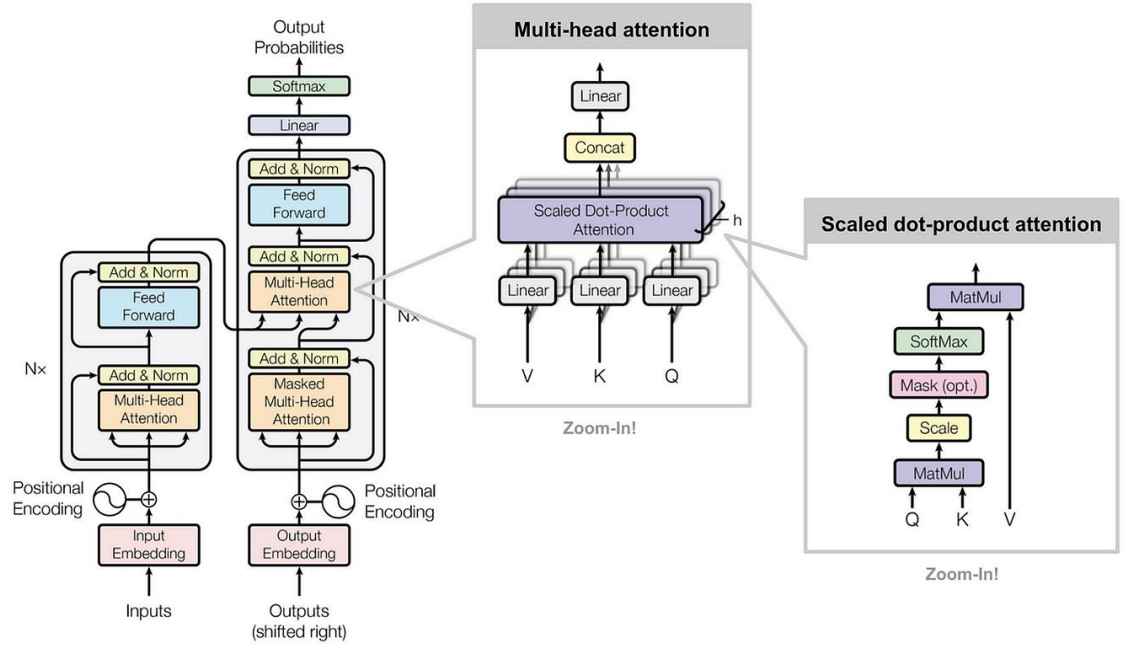
Embeddings

- High-dimensional vector space
- Semantic proximity: Similar concepts cluster together (Cat ≈ Kitten)
- Vector math: King - Man + Woman = Queen.



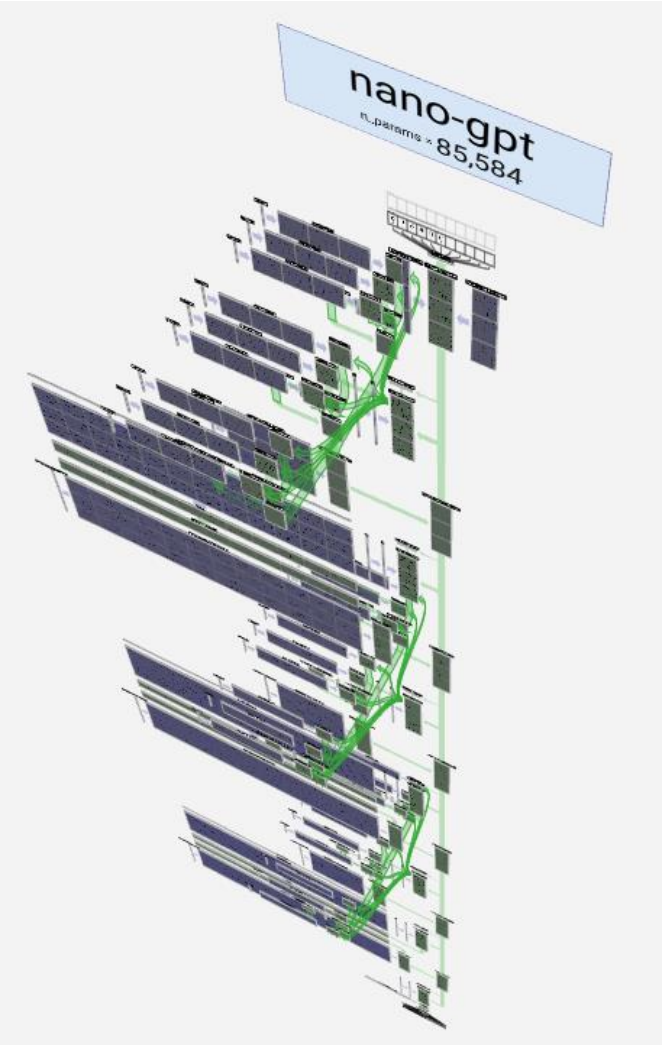
Self-Attention

- Weighting importance: Which words matter most to the current word?
- Disambiguation: Solving "it" or "bank" (river vs. financial)



Transformer Architecture

- Encoder: Understanding and context (BERT)
- Decoder: Predicting and generating (GPT)



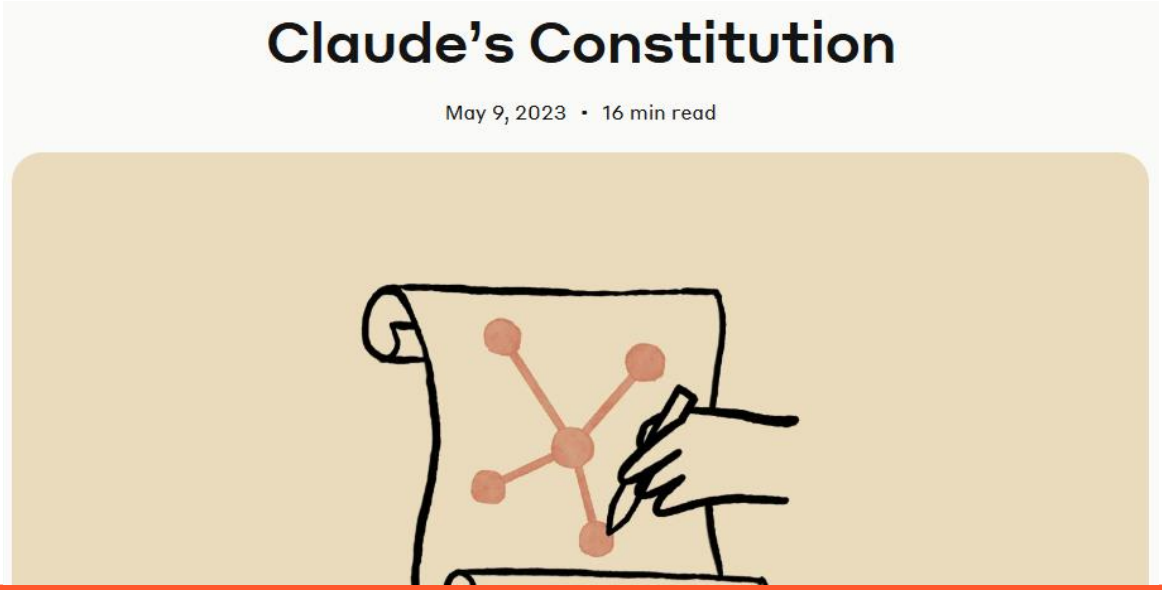
Pre-Training

- Objective: Next Token Prediction
- Data: Common Crawl, Wikipedia, Books, Code
- The result: A "Base Model" that is a powerful mimic but a poor assistant



Post-Training: Instruct and RLHF

- Supervised Fine-Tuning (SFT): Teaching the model to follow commands (Q&A format)
- Reinforcement Learning from Human Feedback (RLHF)
- Alignment: Making models helpful, honest, and harmless



Chat Template & System Prompt

- Most modern models use a **chat template** to structure the data
- Example (ChatML format):
- ```
  <|im_start|>system You are a helpful assistant.<|im_end|>
  <|im_start|>user Explain gravity in simple terms.<|im_end|>
  <|im_start|>assistant Gravity is the force that pulls
  objects toward each other... <|im_end|>
  ```


  ```

Content

- Intro
- LLMs Overview
- Prompting
- Memory & Tools
- Lab



What is a Prompt?

- A prompt is any instruction, question, or input you give to a generative AI model to produce an output
- Outputs can be: text, images, video, audio, code
- **The quality of your output depends heavily on the quality of your input**
- Prompting is a skill, and like any skill, it improves with practice and iteration

- Based on: https://www.coursera.org/specializations/prompting-essentials-google



Why Prompting?

- **In everyday use:** good prompts saves you time
- **In professional contexts**: your prompt determines whether AI output is safe to use. You are accountable for what you share, regardless of what the AI produced
- **In research and analysis**: bad prompts introduce inconsistency (run the same task ten times -> you get ten different interpretations). Precision makes results reproducible
- **In software and product development**: prompts embedded in applications run thousands of times, on inputs you never anticipated, without a human checking each result
- **In agentic AI systems:**
  - A bad prompt runs automatically, at scale, with no human in the loop
  - Agent outputs are often passed directly to other systems
  - Multi-agent systems: ambiguity compounds: bad instruction in step 1 -> worse output in step 2



Zero-, One-, and Few-Shot Prompting

Zero-shot: no examples, just an instruction. The model relies entirely on what it learned during training

- *"Classify the sentiment of this review as positive, negative, or neutral: 'The battery life is terrible, but the screen is gorgeous'"*

One-shot: one example to demonstrate the pattern you want

- *"Classify sentiment like this: 'The coffee was cold and bitter.' → negative. Now classify: 'The battery life is terrible, but the screen is gorgeous'"*

Few-shot: several examples, used when the task is ambiguous, the format is unusual, or you need consistent output style across many items

- *"Here are three examples of how I want bug reports formatted: [example 1] [example 2] [example 3]. Now format this raw issue description the same way: [new input]"*



The TCREI Framework

A five-part structure for writing effective prompts:

- **Task**: What do you want the AI to do?
  - **Persona**: "I am …", "Act as …"
  - **Format**: "Make it sound professional", "Organize it as a table"
- **Context**: What background information is relevant?
- **References**: Can you provide examples? (One-Shot/Few-Shot)
- **Evaluate**: Is the output what you actually wanted?
- **Iterate**: Refine and improve your prompt

"Always Be Iterating": the most important habit in prompting

Task - Persona - Format - Context - References



TCREI Example

- **Weak prompt:** "Write me a work email about a schedule change"
- **Strong prompt (TCREI):** "I'm a gym manager and we have a new gym schedule. Write an email informing our staff of the new schedule. Highlight the fact that the Monday/Wednesday/Friday Hot Yoga class changed from 9:00am to 10:00am. Make the email professional and friendly, and short so readers can skim it quickly"
- **Why it works:** The AI knows who is speaking, why, what to emphasize, and how to format it

Task - Persona - Format - Context - References



Four Ways to Iterate

When your first output isn't right, try one of these:

- **Revisit the framework:** did you leave out context or a format instruction?
- **Break it up:** split a complex prompt into shorter, cleaner sentences
- **Rephrase or reframe:** instead of "write a marketing plan," try "write a story about how this product fits into the daily life of a college student"
- **Add constraints:** narrow the focus: "write only the subject line," "limit to 3 bullet points," "use no jargon"

Task - Persona - Format - Context - References - Constraints



TCREI Example 2

- **Weak prompt:** "Summarize this article"
- **Strong prompt:** "I'm a second-year business student preparing for a seminar discussion. Summarize the attached article on supply chain disruptions. Write the summary in plain, conversational language as if explaining to a curious friend who has no business background. Keep it under 150 words"
- **Key lesson:** Tone and audience instructions matter as much as the task itself

Task - Persona - Format - Context - References - Constraints



TCREI Example Multimodal

Prompt using an image as a reference

- [Upload photo of ingredients in a fridge] "Based on the ingredients visible in this photo, suggest three quick dinner recipes that take under 30 minutes to prepare. Format each recipe as a short bullet list"
- Modern LLMs can accept images, PDFs, audio, code, …

Task - Persona - Format - Context - References - Constraints



Advanced Technique: Prompt Chaining

Prompt chaining breaks a complex goal into a sequence of connected prompts, each building on the last

Example: Marketing a novel:

- *Prompt 1:* "Generate three options for a one-sentence summary of this novel manuscript. The summary should match the voice and tone of the manuscript but be catchier and engaging"
- *Prompt 2:* "Create a tagline combining the best elements of those three summaries, with special focus on the plot twist. Make it concise and leave the reader wanting more"
- *Prompt 3:* "Generate a six-week promotional book tour plan, including recommended locations and marketing channels for each stop"

Why it works: Each prompt is focused. The AI isn't overwhelmed by one giant request



Advanced Technique: Chain of Thought (CoT)

Chain of Thought prompting asks the AI to show its reasoning step by step, rather than jumping straight to an answer

- Simply add: *"Explain your reasoning step by step"* or *"Think through this carefully before answering"*
- **Example:** "A store sells items at a 30% markup over wholesale price. If the retail price of a jacket is $91, what was the wholesale price? Explain your reasoning step by step"
- **Why it matters:** CoT prompting significantly improves accuracy on math, logic, and multi-step problems. It also lets you catch where the model went wrong

(Note: has become a core technique in modern AI systems)



Meta-Prompting

When you're stuck, ask the AI to write the prompt for you:

- **Example:** "I want to use AI to help me prepare for a technical job interview for a software engineering role. I'm not sure how to prompt you effectively for this. Can you suggest three different prompts I could use to get the most useful practice?"
- Then use one of those prompts to actually run the session

Why it works: Meta-prompting helps you discover framings you wouldn't have thought of yourself. It's also a good way to learn prompting by example



Content

- Intro
- LLMs Overview
- Prompting
- Memory & Tools
- Lab



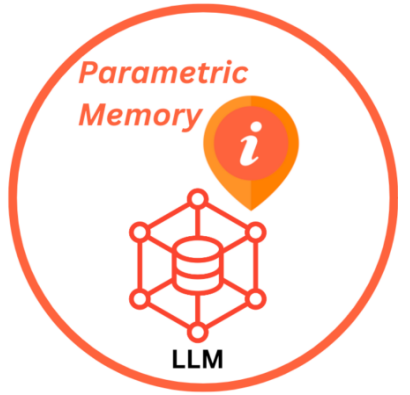
Two Types of Memory in LLMs

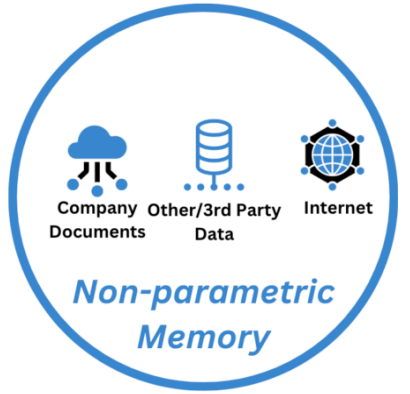
Parametric memory

- Knowledge baked into model weights during pretraining
- Fast at inference, but static (frozen at training cutoff)
- Cannot be updated without retraining or fine-tuning

Non-parametric (external) memory

- Knowledge stored outside the model: in files, databases, APIs
- Retrieved at inference time and injected into the context
- Can be updated, versioned, and audited independently





Retrieval-Augmented Generation (RAG)

Idea: Before the LLM answers, retrieve relevant documents and include them in the prompt

The RAG pipeline in three steps:

1. **Index**: convert your documents into searchable vector representations
2. **Retrieve**: given a user query, find the most relevant chunks
3. **Generate**: pass the query + retrieved chunks to the LLM as context

Why this works:

- The LLM now has the answer in its context window
- You can cite sources and verify answers
- No retraining required: swap documents without touching the model



Function Calling: Step-by-step

1. You provide a list of tool schemas (JSON) alongside the user message
2. The LLM reads the schemas and decides if a tool is needed
3. If yes, instead of a text reply it outputs a structured tool call:
   `{"name": "get_weather", "arguments": {"city": "Vienna"}}`
4. Your code intercepts this, runs the function, gets the result
5. You append the result to the conversation and call the LLM again
6. The LLM now has the result in context and generates a final text reply



Functi

1. You pro
2. The LL
3. If yes, in
   {"name
4. Your co
5. You app
6. The LL

ssage

nna"}}

reply

```
<|im_start|>system
# Tools
<tools>
{"type": "function", "function": {"name": "get_weather", "description": "Get weather", "parameters": {"type": "object",
"properties": {"city": {"type": "string"}}}}}
{"type": "function", "function": {"name": "get_flight_status", "description": "Check flight status", "parameters": {"type":
"object", "properties": {"flight_number": {"type": "string"}}}}}
</tools><|im_end|>
<|im_start|>user
Should I pack an umbrella for my UA902 flight to Tokyo?<|im_end|>
<|im_start|>assistant
<think>
User is asking about packing an umbrella for flight UA902 to Tokyo.
I need to:
1. Check the flight status to confirm it's going to Tokyo
2. Check Tokyo's weather to see if rain is expected
Let me start with the flight status.
</think>

I'll help you decide if you need an umbrella. Let me check your flight details and the weather in Tokyo.

<tool_call>
{"name": "get_flight_status", "arguments": {"flight_number": "UA902"}}
</tool_call><|im_end|>
<|im_start|>user
<tool_response>
 {"flight": "UA902", "destination": "Tokyo (NRT)", "status": "On time", "departure": "14:30"}
</tool_response><|im_end|>
<|im_start|>assistant
<think>
Flight confirmed to Tokyo, departing at 14:30. Now checking weather.
</think>

<tool_call>
{"name": "get_weather", "arguments": {"city": "Tokyo"}}
</tool_call><|im_end|>
<|im_start|>user
<tool_response>
 {"temperature": 18, "condition": "rainy", "precipitation": "80%"}
</tool_response><|im_end|>
<|im_start|>assistant
<think>
Tokyo weather shows rainy conditions with 80% precipitation. Definitely need an umbrella.
</think>

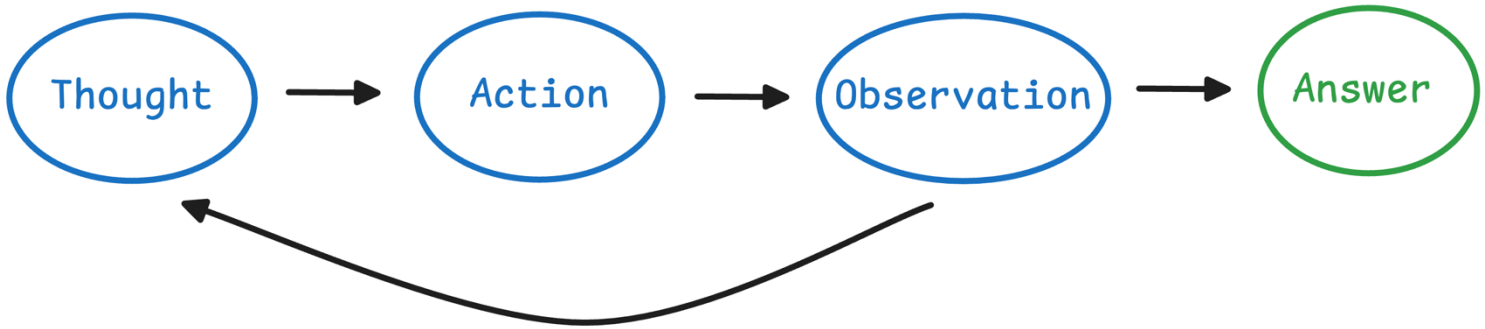
Yes, you should definitely pack an umbrella! Your UA902 flight to Tokyo is on time (departing at 14:30), and Tokyo is currently
experiencing rainy weather with an 80% chance of precipitation. Better to be prepared! ☔<|im_end|>
```

The ReAct Pattern

ReAct = Reasoning + Acting (Yao et al., 2022)

The core loop:

- *Thought:* I need to find the current price of product X.
- *Action:* search_database(query="product X price")
- *Observation:* {"name": "Product X", "price": 49.99, "in_stock": true}
- *Thought:* The price is $49.99 and it's in stock. I can answer the user.
- *Action:* respond("Product X costs $49.99 and is currently in stock.")





Hierarchy

- **LLM:** A standalone, stateless "reasoning engine" that predicts the next token based on a prompt
- **AI Workflow:** A fixed, linear chain of prompts where the developer defines every step and the LLM simply processes the data at each junction
- **AI Agent:** A system that uses a reasoning loop to dynamically choose and execute tools to achieve a specific (short-term goal)
- **Agentic AI:** A broader design where AI systems possess high levels of autonomy to plan, self-correct, and navigate complex objectives
- **Swarms:** A decentralized network of specialized agents that communicate and hand off tasks to one another to solve large problems



Content

- Intro
- LLMs Overview
- Prompting
- Memory & Tools
- Lab



Lab

- Discussion: What do you want to learn?
- Jupyter Notebook: set up the API



ขอบคุณครับ

Thank you!

etienne@cmkl.ac.th

